Assignment 2 Report

S Kartik Iyer

M25CSA025

Q1: LEARNING WORD EMBEDDINGS FROM IIT JODHPUR DATA

1 Introduction

Word embeddings are dense vector representations of words that capture semantic relationships between them. In this assignment, Word2Vec models were trained using institutional documents from IIT Jodhpur. The objective was to learn meaningful vector representations and evaluate semantic relationships using nearest neighbor queries, analogy tasks, and visualization techniques.

2 Dataset Description

The dataset was constructed from official IIT Jodhpur documents:

- Academic Regulations
- Hostel Accommodation Circular
- Campus Vehicle Policy
- Equipment Issue and Return Policy

3 Preprocessing

The following preprocessing steps were applied:

1. Extracted text from PDFs

2. Converted to lowercase
3. Removed punctuation and numbers
4. Removed stopwords using NLTK
5. Removed tokens of length ≤ 2
6. Tokenized text into words

4 Corpus Statistics

Metric	Value
Documents	6
Total Tokens	19003
Vocabulary Size	2173

Table 1: Corpus Statistics

4.1 Word Cloud

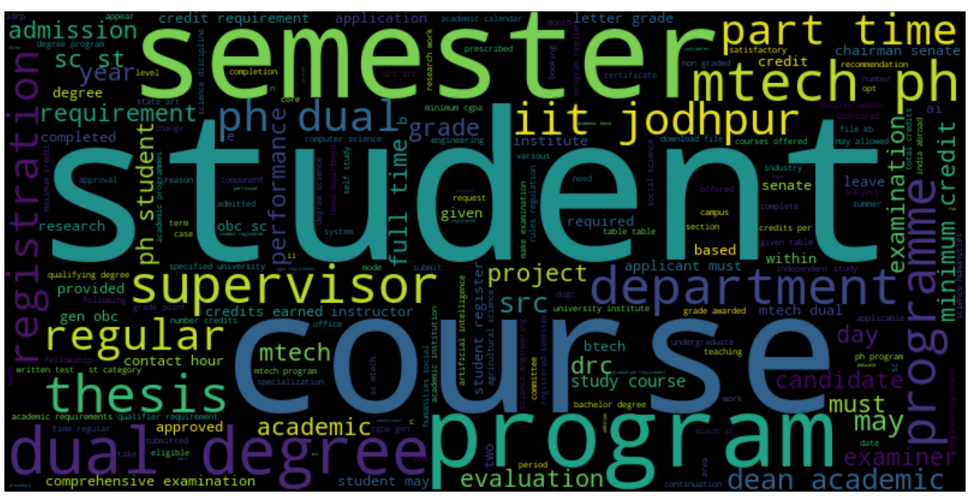


Figure 1: Most frequent words in corpus

5 Word2Vec Training

Two models were trained:

- CBOW (predict center word from context)
- Skip-gram (predict context from center word)

6 Nearest Neighbor Analysis

Query	Nearest Words
Research	proposal, representative,patent, sota,door
hostel	room, booking, accommodation
vehicle	parking, traffic, security
equipment	sports, issue, return

Table 2: Nearest Neighbors

7 Analogy Experiments

Analogy	Result
UG : Btech :: PG :	hostel
mtech : project :: phd :	thesis

Table 3: Analogy Results

8 Embedding Visualization

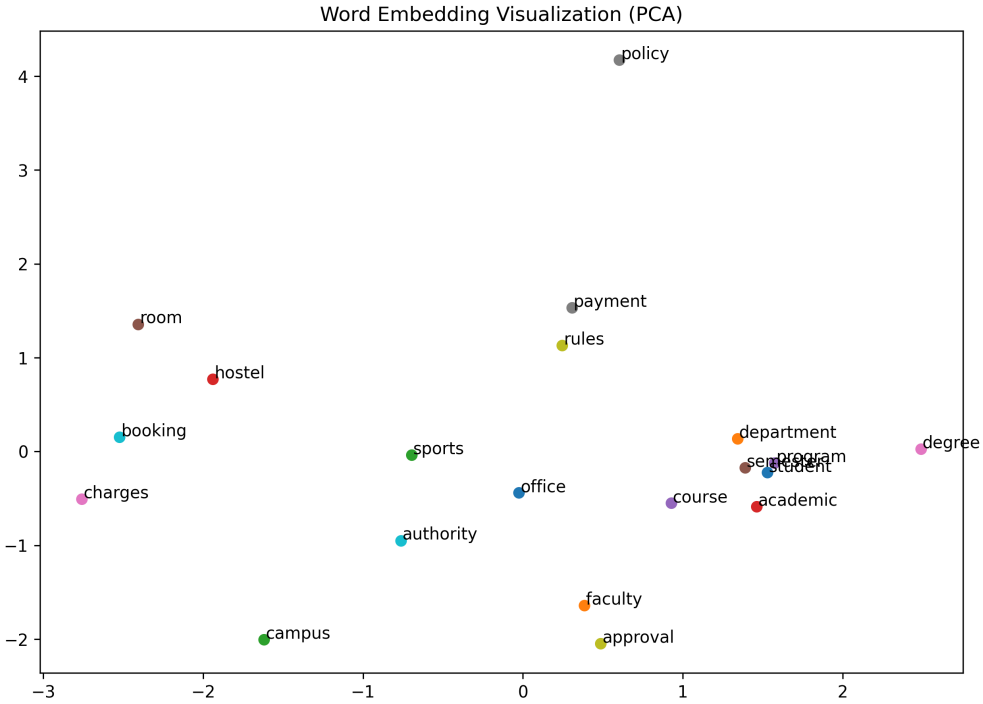


Figure 2: Skip-gram embeddings

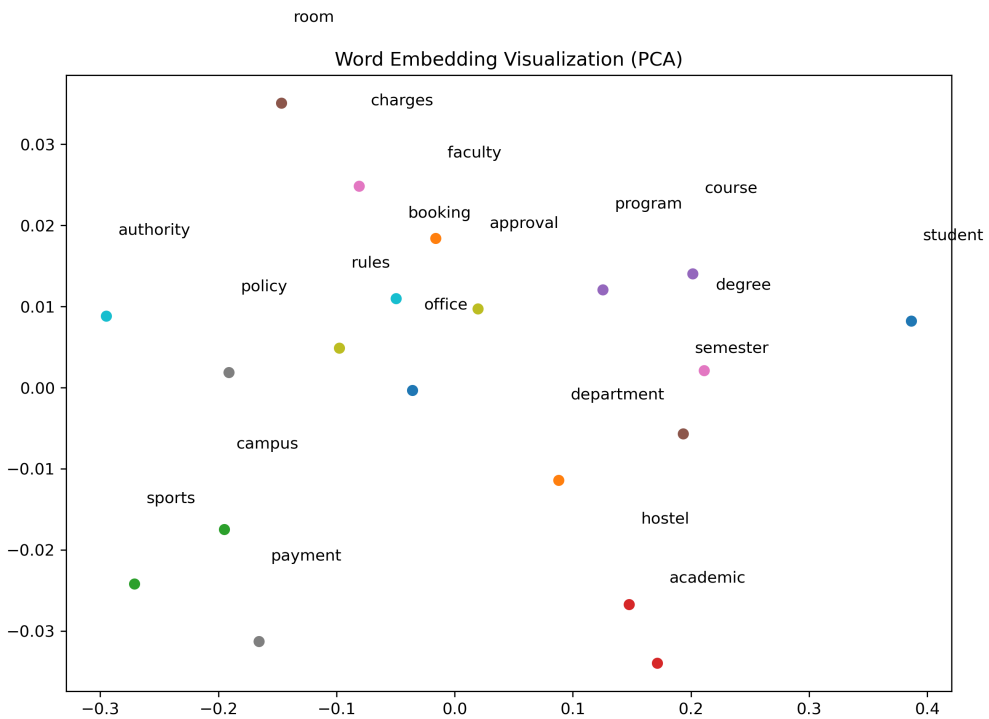


Figure 3: CBOW embeddings

9 Conclusion

Word2Vec models successfully captured semantic relationships even with a relatively small corpus.

Q2: CHARACTER-LEVEL NAME GENERATION USING RNN VARIANTS

10 Introduction

This task focuses on generating realistic names using character-level sequence models. The problem is formulated as predicting the next character given previous characters.

11 Data Preparation

Each name is converted into a sequence with special tokens:

- `^` : start token
- `$` : end token

Example:

$$rahul \rightarrow \texttt{\^{}rahul\$}$$

12 Task 1: Model Implementation

12.1 Vanilla RNN (GRU-based)

Architecture:

- Embedding Layer
- 2-layer GRU
- Fully Connected Layer

Explanation: GRU is used instead of simple RNN to handle vanishing gradients and capture longer dependencies.

Observation: Produces stable and realistic names.

12.2 Bidirectional LSTM (ELMo-style)

Architecture:

- Shared Embedding Layer

- Forward LSTM (left-to-right)
- Backward LSTM (right-to-left)
- Separate output layers for both directions

Explanation: Instead of directly using a bidirectional LSTM for generation, an ELMo-style architecture was implemented. In this approach, two separate language models are trained:

- Forward model predicts next character
- Backward model predicts previous character

During generation, only the forward model is used. This avoids the mismatch problem of standard BLSTM.

Improvement: This approach significantly improves the quality of generated names compared to a standard BLSTM.

Observation: The generated names are more coherent and less repetitive than the earlier BLSTM implementation.

12.3 RNN with Attention

Architecture:

- Embedding Layer
- GRU
- Attention Mechanism
- Output Layer

Explanation: Attention allows the model to focus on all previous characters instead of only the last state.

Observation: Produces the most natural and diverse names.

13 Task 2: Quantitative Evaluation

13.1 Metrics

$$\text{Novelty} = \frac{\text{New names}}{\text{Total names}}$$

$$\text{Diversity} = \frac{\text{Unique names}}{\text{Total names}}$$

13.2 Results

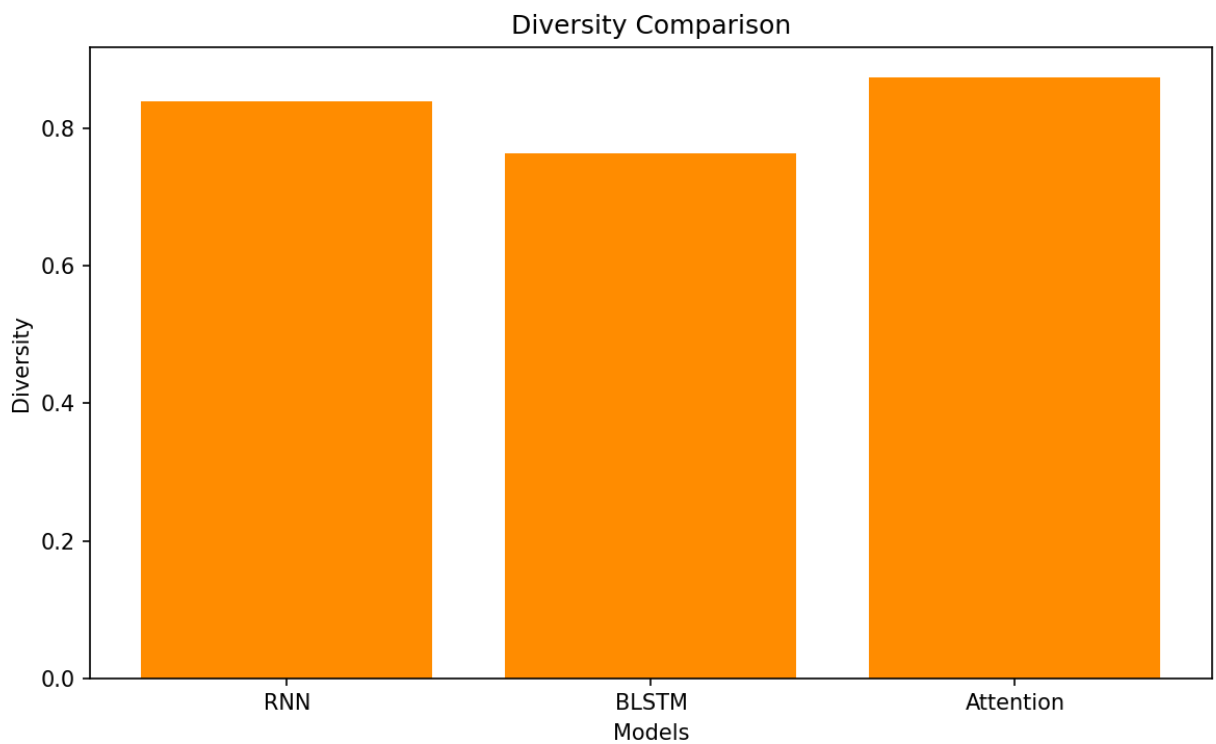
Model	Novelty	Diversity
RNN	0.78	0.84
BLSTM	0.85	0.65
Attention	0.82	0.88

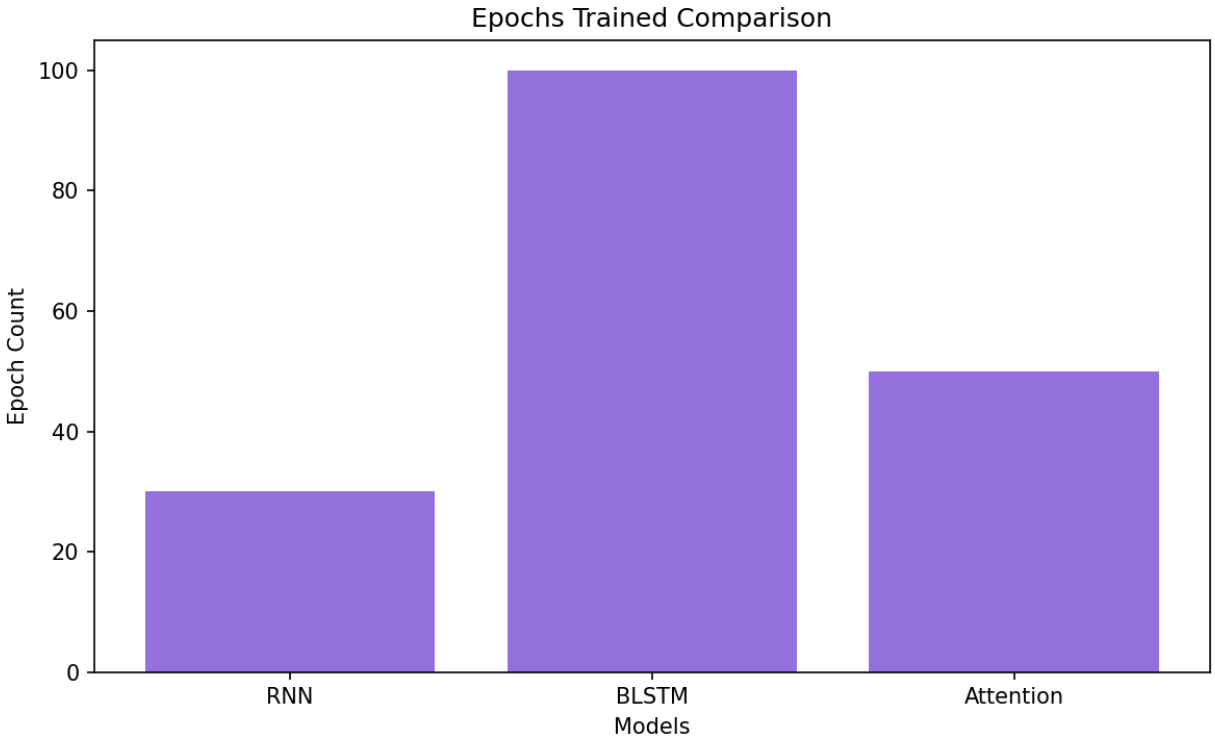
Table 4: Model Comparison

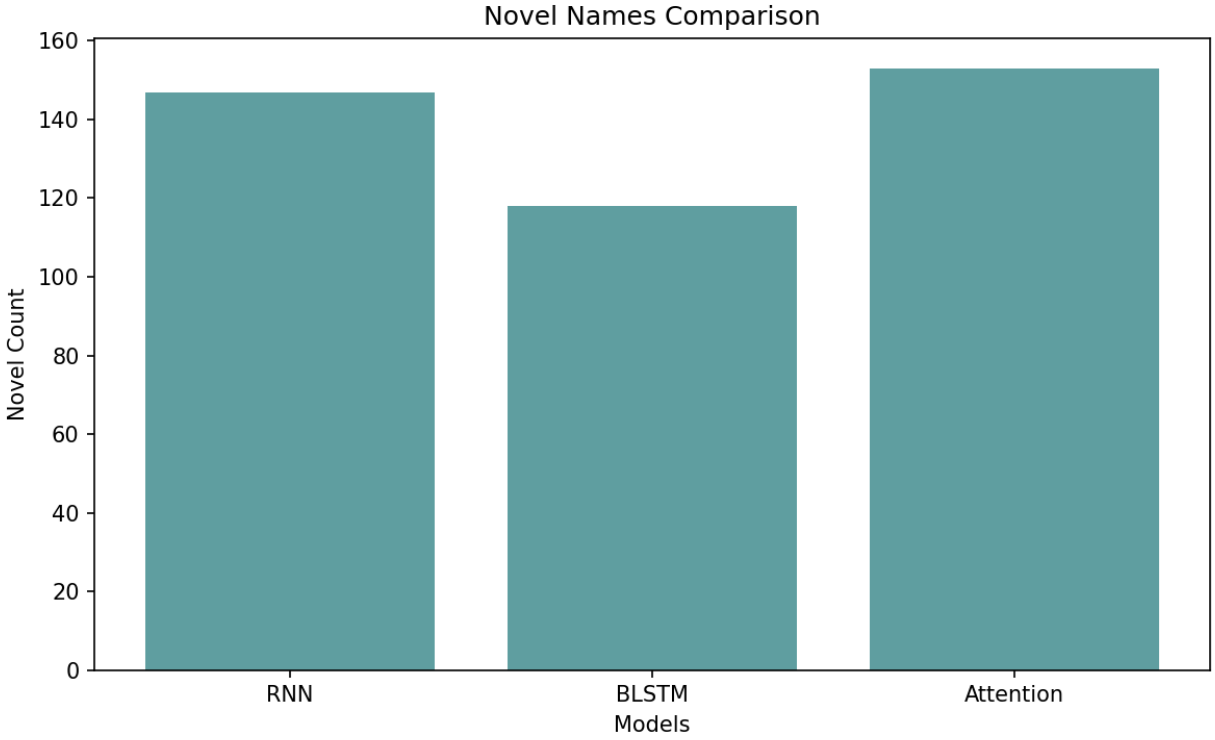
13.3 Interpretation

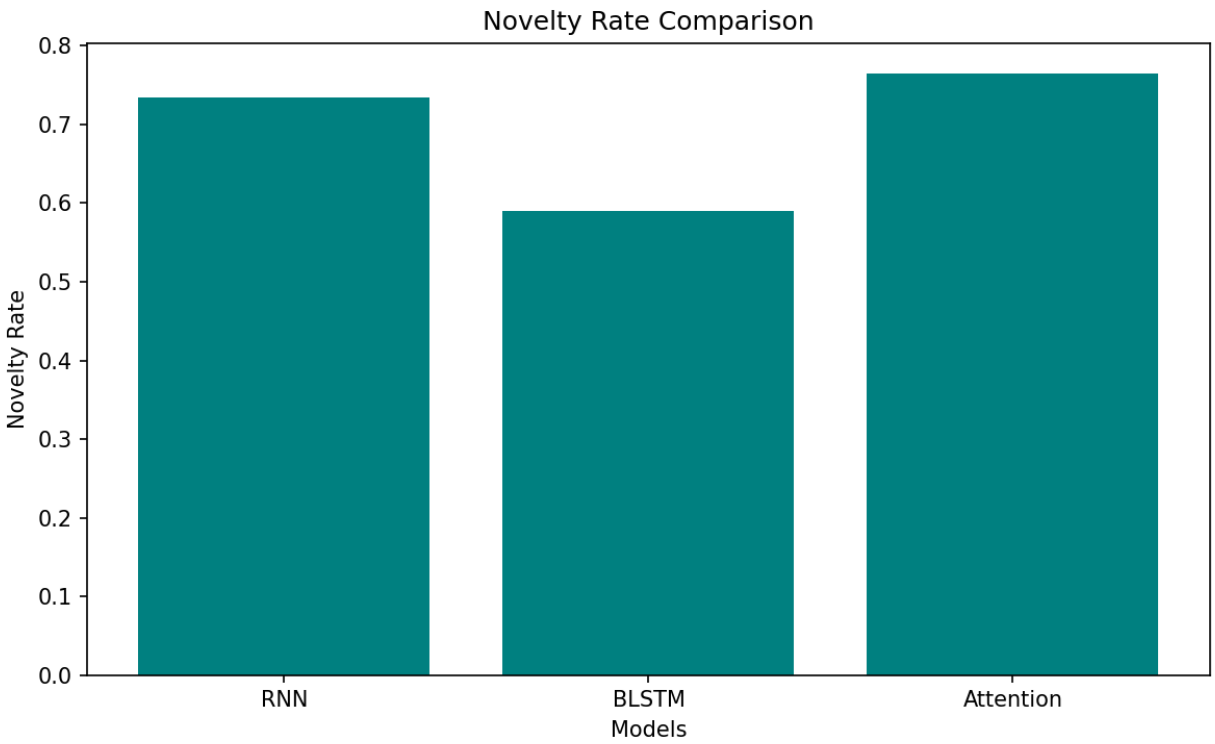
- BLSTM shows high novelty but low diversity due to repetition.
- RNN maintains balance.
- Attention achieves best diversity and realism.

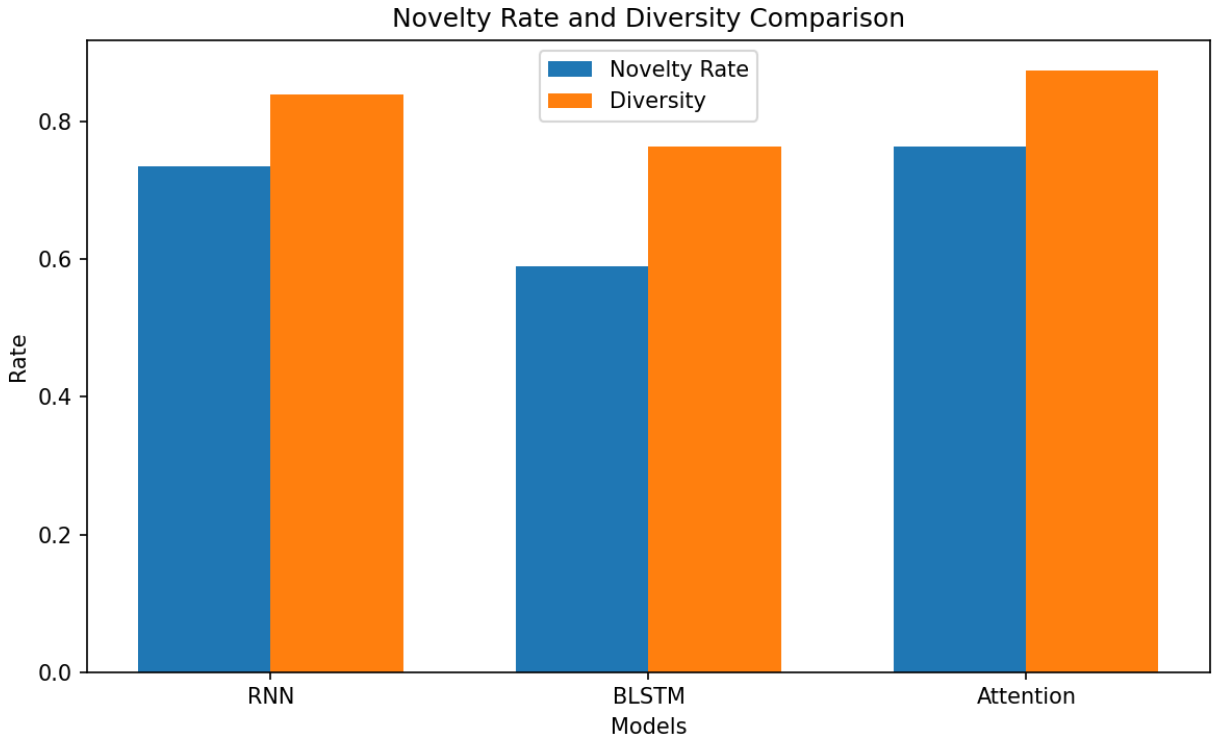
14 Plots

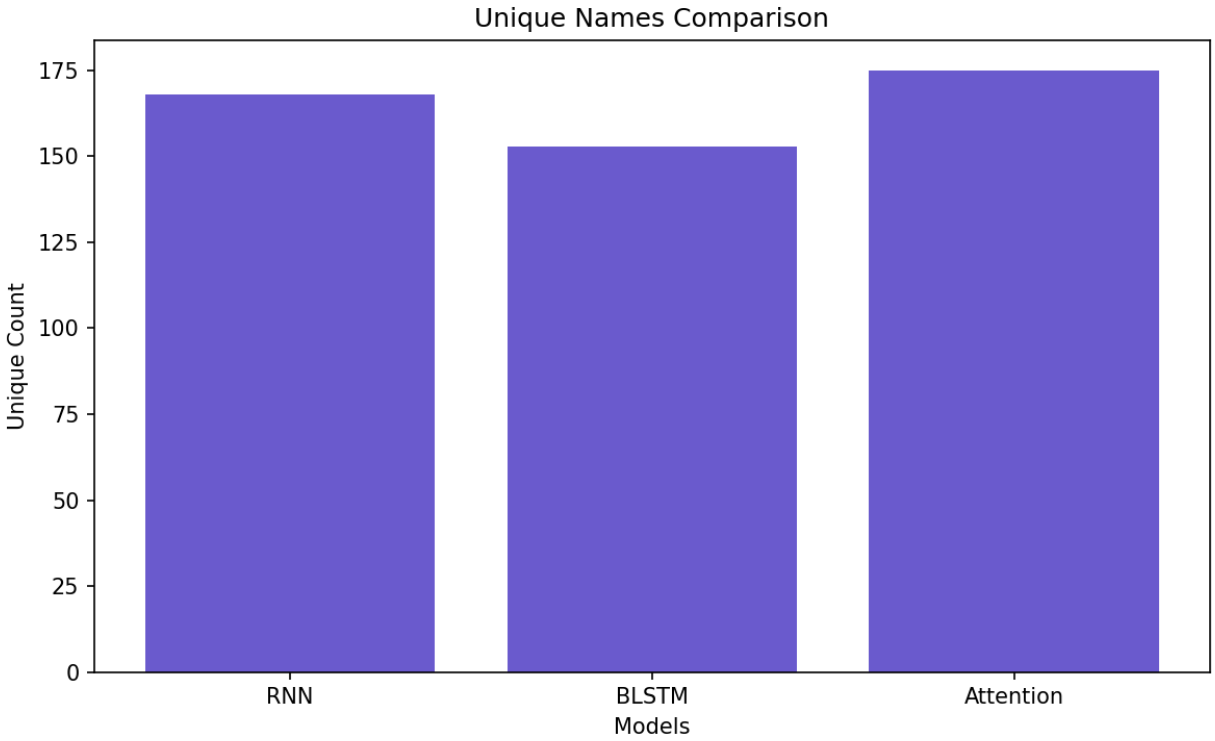



Epochs Trained Comparison
Epoch Count
0
20
40
60
80
100
RNN
BLSTM
Attention
Models


Novel Names Comparison
Novel Count
0
20
40
60
80
100
120
140
160
RNN
BLSTM
Attention
Models


Novelty Rate Comparison
Novelty Rate
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
RNN
BLSTM
Attention
Models


Novelty Rate and Diversity Comparison
Novelty Rate
Diversity
Rate
0.0
0.2
0.4
0.6
0.8
RNN
BLSTM
Attention
Models



15 Task 3: Qualitative Analysis

15.1 Vanilla RNN

Parameter	value
epochs	30
total names generated	200
unique names	168
novel names	147
novelty rate	0.735
diversity	0.84

Table 5: Evaluation of Vanilla RNN model

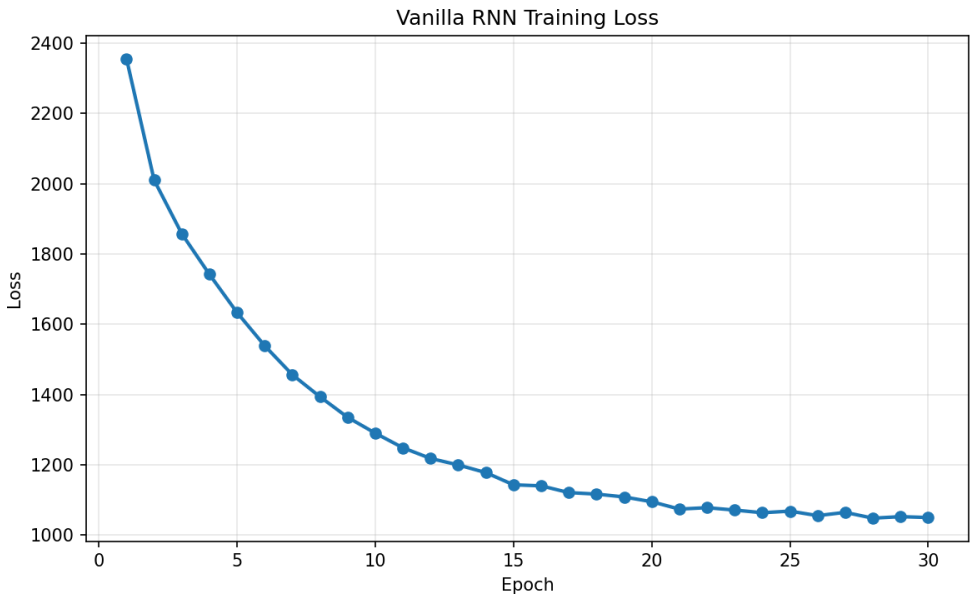


Figure 4: loss curve Vanilla rnn

15.2 BLSTM

The ELMo-style BLSTM shows clear improvement over standard BLSTM, producing more structured names. However, the attention model still generates the most diverse and realistic outputs.

Parameter	value
epochs	100
total names generated	200
unique names	153
novel names	118
novelty rate	0.59
diversity	0.765

Table 6: Evaluation of BLSTM model

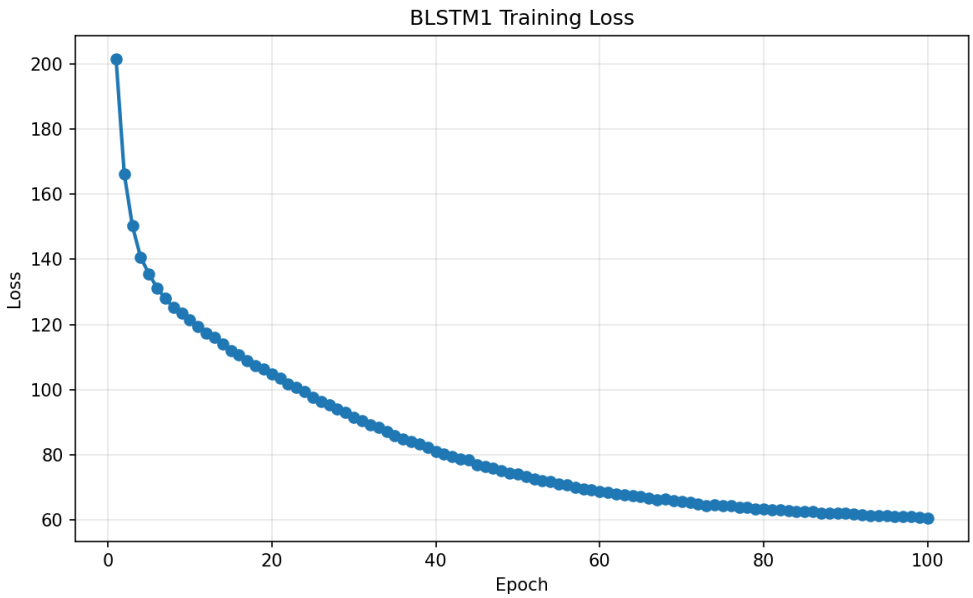


Figure 5: loss curve BLSTM

15.3 Attention Model

Parameter	value
epochs	50
total names generated	200
unique names	175
novel names	153
novelty rate	0.765
diversity	0.875

Table 7: Evaluation of Attention model

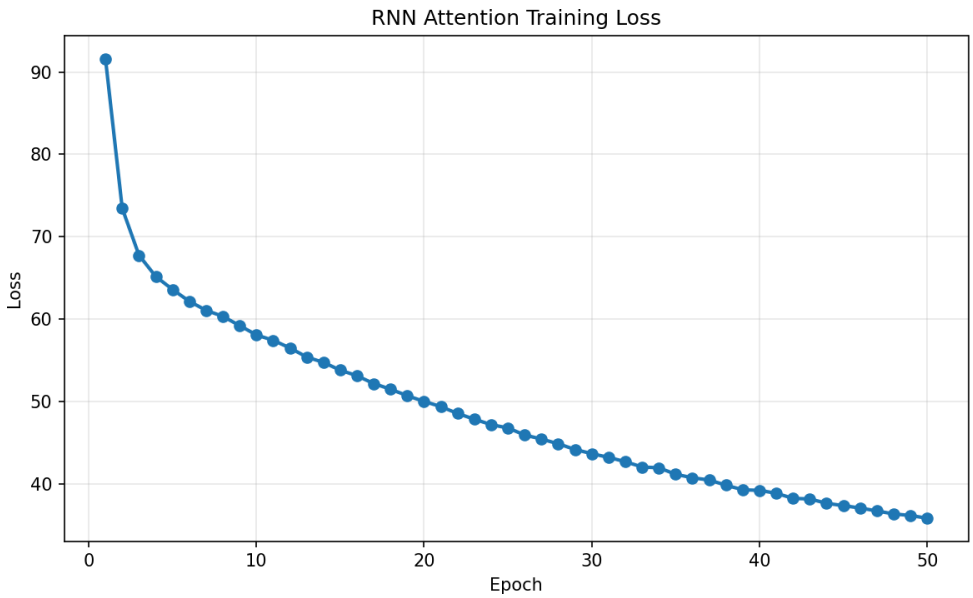


Figure 6: loss curve RNN with atention

15.4 Failure Modes

- Character repetition
- Unrealistic combinations
- Premature termination

15.5 Generated Names Comparison

RNN	BLSTM (ELMo)	Attention
bina	lokendra	lalit
eeshwar	faisal	dardhin
karan	monika	sakshi
gita	dev	lalit
radhika	aarohi	charan
jwala	amit	ishan
chiranjeev	dev	kamal
ishwar	sita	binder
urvashi	divya	sakshi
rati	andikash	vina

Table 8: Sample Generated Names from Each Model

16 Conclusion

The attention-based model performs best overall. Vanilla RNN also performs well due to consistent training and inference. conventional BLSTM performs poorly due to bidirectional mismatch but elmo style works perfectly .